

=====

=====

SentinelAI Enterprise SOC Platform – Complete Source Code Bundle

System Architect & Developer: Raj Sen (Mark 1 Systems Architect)

TABLE OF CONTENTS

MAIN APPLICATION ENTRY POINTS & UTILITIES

SentinelAI/dashboard/app.py
SentinelAI/dashboard/utils.py
SentinelAI/dashboard/styles.css

DASHBOARD UI COMPONENTS & CARDS

SentinelAI/dashboard/components/copilot_card.py
SentinelAI/dashboard/components/incident_storyline_card.py
SentinelAI/dashboard/components/behavioral_identity_card.py
SentinelAI/dashboard/components/risk_engine_card.py
SentinelAI/dashboard/components/timeline.py
SentinelAI/dashboard/components/analyst_console.py
SentinelAI/dashboard/components/incident_details.py
SentinelAI/dashboard/components/overview.py
SentinelAI/dashboard/components/threat_intel.py
SentinelAI/dashboard/components/live_stream.py
SentinelAI/dashboard/components/explainability_panel.py
SentinelAI/dashboard/components/analytics.py

DETECTION ENGINES & AI COPILOT

SentinelAI/detection/ai_analyst_copilot.py
SentinelAI/detection/detector.py
SentinelAI/detection/autoencoder.py
SentinelAI/detection/isolation_forest.py
SentinelAI/detection/behavioral_identity.py

SentinelAI/detection/dynamic_risk_engine.py
SentinelAI/detection/incident_storyline.py
SentinelAI/detection/evaluator.py

BACKEND REST API & PERSISTENCE SERVICES

SentinelAI/backend/app.py
SentinelAI/backend/services/incident_persistence.py
SentinelAI/backend/services/incident_service.py
SentinelAI/backend/services/pipeline_service.py
SentinelAI/backend/api/alerts.py
SentinelAI/backend/api/analytics.py
SentinelAI/backend/api/dashboard.py
SentinelAI/backend/api/feedback.py
SentinelAI/backend/api/incidents.py

FEATURE ENGINEERING, CLASSIFIER & EXPLAINABILITY

SentinelAI/feature_engine/event_features.py
SentinelAI/feature_engine/feature_pipeline.py
SentinelAI/feature_engine/preprocessing.py
SentinelAI/feature_engine/window_features.py
SentinelAI/classifier/classifier.py
SentinelAI/classifier/evaluate_classifier.py
SentinelAI/classifier/inference_classifier.py
SentinelAI/classifier/models_comparison.py
SentinelAI/explainer/explainability.py
SentinelAI/explainer/recommendations.py
SentinelAI/explainer/risk_engine.py
SentinelAI/explainer/shap_engine.py

DATA GENERATION, CLI SCRIPTS & DEPLOYMENT CONFIG

SentinelAI/data_generator/attack_injector.py
SentinelAI/data_generator/entity_templates.py
SentinelAI/data_generator/generator.py

```
SentinelAI/data_generator/schemas.py
SentinelAI/scripts/generate_data.py
SentinelAI/scripts/run_backend.py
SentinelAI/scripts/run_dashboard.py
SentinelAI/scripts/run_explainability.py
SentinelAI/scripts/run_feature_engineering.py
SentinelAI/scripts/train_anomaly_detector.py
SentinelAI/scripts/train_attack_classifier.py
SentinelAI/config/data_config.yaml
SentinelAI/Dockerfile
SentinelAI/docker-compose.yml
```

SentinelAI/requirements.txt

```
=====
=====
```

FILE PATH: SentinelAI/dashboard/app.py

```
"""
```

```
SentinelAI – Enterprise Security Operations Center (SOC) Dashboard
```

```
Main Streamlit Application Entry Point.
```

```
Incident-Centric UEBA & SOC Decision Support Platform.
```

```
Persistent Analyst Feedback & Audit Logging Integration.
```

```
Vanilla Light Mode Theme System.
```

```
"""
```

```
import os
```

```
import sys
```

```
import streamlit as st
```

```
import pandas as pd
```

Add project root to path

```
PROJECT_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(file)))
```

```
if PROJECT_ROOT not in sys.path:
```

```
sys.path.insert(0, PROJECT_ROOT)
```

```
from dashboard.utils import load_dataset, load_models
```

```
from dashboard.components.overview import render_overview
```

```
from dashboard.components.threat_intel import render_threat_intel
```

```

from dashboard.components.explainability_panel import
render_explainability_panel
from dashboard.components.timeline import render_timeline
from dashboard.components.analytics import render_analytics
from dashboard.components.analyst_console import render_analyst_console
from dashboard.components.incident_details import
render_incident_details

```

Page Config

```

st.set_page_config(
page_title="SentinelAI - SOC Security Console",
layout="wide",
initial_sidebar_state="collapsed",
)

```

Load Custom CSS (Vanilla Light Mode)

```

css_path = os.path.join(PROJECT_ROOT, "dashboard", "styles.css")
if os.path.exists(css_path):
with open(css_path) as f:
st.markdown(f"", unsafe_allow_html=True)

```

```

def main():
# Load Data & Models from CSV Single Source of Truth
df_raw, df_proc = load_dataset()
detector, classifier, explainer = load_models()

```

```

# Calculate active anomalies directly from persisted CSV dataset
anomalies = df_proc[df_proc["label"] == "anomaly"]
crit = len(anomalies[anomalies["attack_type"].isin(["lateral_movement",
high = len(anomalies[anomalies["attack_type"].isin(["brute_force", "imp

st.markdown(
    f""
    <div class="soc-header-banner">
        <div>
            <h1>SentinelAI</h1>
            <p>Enterprise Behavioral Anomaly Detection & Incident-Centr
            <div style="margin-top: 6px;">
                <span class="developer-watermark">Mark 1 Architect & De
            </div>

```

```

        </div>
        <div style="display: flex; gap: 8px; align-items: center; margin: 0 auto; width: 100%; text-align: center;">
            <span class="badge-critical">Critical: {crit}</span>
            <span class="badge-high">High: {high}</span>
            <span class="badge-low">System: Operational</span>
        </div>
    </div>
    """
    unsafe_allow_html=True,
)

# Streamlined Top Navigation Bar
page = st.radio(
    "Navigation",
    [
        "Overview",
        "Threat Intelligence",
        "Incident Details",
        "Explainable AI (XAI)",
        "User Behavior Timeline",
        "Analyst Console",
    ],
    horizontal=True,
    label_visibility="collapsed",
)

# Render Selected Page
if "Overview" in page:
    render_overview(df_proc)
    render_analytics(df_proc, theme_mode="Light Mode")
elif "Threat Intelligence" in page:
    render_threat_intel(df_proc)
elif "Incident Details" in page:
    render_incident_details(df_proc)
elif "Explainable AI" in page:
    render_explainability_panel(df_proc, explainer)
elif "User Behavior" in page:
    render_timeline(df_proc)
elif "Analyst Console" in page:

```

```

        render_analyst_console(df_proc)

# Footer Developer Credit
st.markdown(
    """
    <div class="footer-credit">
        <b>Raj Sen | MARK 1</b>
    </div>
    """,
    unsafe_allow_html=True,
)

```

```

if name == "main":
    main()

```

```

=====

```

```

=====

```

FILE PATH: SentinelAI/dashboard/utils.py

```

"""

```

SentinelAI Dashboard – Utility & Data Integration Helpers

Loads processed datasets, raw events, trained anomaly detector, attack classifier, and explainability engine.

```

"""

```

```

import os
import sys
import logging
from typing import Tuple, Dict, Any, Optional

```

```

import pandas as pd
import numpy as np
import streamlit as st

```

```

PROJECT_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(file)))
if PROJECT_ROOT not in sys.path:
    sys.path.insert(0, PROJECT_ROOT)

```

```

from detection.detector import AnomalyDetector
from classifier.classifier import AttackClassifier
from explainer.explainability import ExplainabilityEngine

```

```

from backend.services.incident_persistence import
ensure_persistence_schema

logger = logging.getLogger(name)

def load_dataset() -> Tuple[pd.DataFrame, pd.DataFrame]:
    """Load raw events and engineered dataset directly from CSV file on
    disk."""
    raw_path = os.path.join(PROJECT_ROOT, "data", "raw", "events.csv")
    proc_path = os.path.join(PROJECT_ROOT, "data", "processed",
    "engineered_dataset.csv")

    if not os.path.exists(proc_path):
        df_raw, df_proc = _generate_mock_dataset()
        df_proc = ensure_persistence_schema(df_proc)
        os.makedirs(os.path.dirname(proc_path), exist_ok=True)
        df_proc.to_csv(proc_path, index=False)
        return df_proc, df_proc

    df_raw = pd.read_csv(raw_path) if os.path.exists(raw_path) else pd.Data
    df_proc = pd.read_csv(proc_path)
    df_proc = ensure_persistence_schema(df_proc)

    return df_raw, df_proc

```

```

@st.cache_resource
def load_models() -> Tuple[Optional[AnomalyDetector],
Optional[AttackClassifier], Optional[ExplainabilityEngine]]:
    """Load backend machine learning models."""
    model_dir = os.path.join(PROJECT_ROOT, "models")
    if not os.path.exists(os.path.join(model_dir, "detector_meta.pkl")):
        return None, None, None

```

```

try:
    detector = AnomalyDetector().load(model_dir)
    classifier = AttackClassifier().load(os.path.join(model_dir, "attac
    explainer = ExplainabilityEngine(detector, classifier)
    return detector, classifier, explainer
except Exception as e:

```

```
logger.error(f"Failed to load models: {e}")
return None, None, None
```

```
=====
=====
```

FILE PATH:

SentinelAI/dashboard/components/copilot_card.py

```
"""
```

```
AI Analyst Copilot Card Dashboard Component.
```

Renders an enterprise investigation brief using native Streamlit UI components to ensure zero raw HTML markup, tags, or unformatted text bleed into the dashboard.

```
"""
```

```
import streamlit as st
```

```
from typing import Dict, Any
```

```
def render_copilot_card(copilot_brief: Dict[str, Any]):
```

```
    """Render the AI Analyst Copilot investigation brief as a clean enterprise SOC report."""
```

```
    entity_id = copilot_brief.get("entity_id", "Unknown")
```

```
    exec_summary = copilot_brief.get("executive_summary", "")
```

```
    why_detected = copilot_brief.get("why_detected", "")
```

```
    evidence = copilot_brief.get("supporting_evidence", [])
```

```
    risk_exp = copilot_brief.get("risk_score_explanation", "")
```

```
    biz_impact = copilot_brief.get("business_impact", "")
```

```
    rec_actions = copilot_brief.get("recommended_actions", [])
```

```
    mitre = copilot_brief.get("mitre_attack_mapping", {})
```

```
    soc_resp = copilot_brief.get("suggested_soc_response", {})
```

```
    time_saved = copilot_brief.get("estimated_investigation_time_saved", "")
```

```
    confidence = copilot_brief.get("copilot_confidence_pct", 95.0)
```

```
# Top Header Box
```

```
st.markdown("### SentinelAI Analyst Copilot Brief")
```

```
st.caption(f"Target Entity: **{entity_id}** • Confidence: **{confidence
```

```
st.markdown("---")
```

```

# Executive Summary & Why Detected Sections
st.markdown("#### Executive Summary")
st.info(exec_summary)

st.markdown("#### Why Detected")
st.warning(why_detected)

col1, col2 = st.columns(2)

with col1:
    st.markdown("#### MITRE ATT&CK Mapping")
    st.markdown(f"- **Tactic:** {mitre.get('tactic', 'Initial Access')}")
    st.markdown(f"- **Technique:** {mitre.get('technique_id', 'T1078')}")

    st.markdown("#### Business Impact")
    st.error(biz_impact)

    st.markdown("#### Risk Explanation")
    st.markdown(f"> {risk_exp}")

    st.markdown("#### Supporting Evidence Chain")
    for ev in evidence:
        st.markdown(f"- {ev}")

with col2:
    st.markdown("#### Recommended SOC Containment Steps")
    for act in rec_actions:
        st.markdown(f"- {act}")

    st.markdown("#### Executable Remediation Command (PowerShell)")
    ps_cmd = soc_resp.get("powershell", f"Disable-LocalUser -Name '{ent}")
    st.code(ps_cmd, language="powershell")

    aws_cmd = soc_resp.get("aws_cli", "")
    if aws_cmd:
        st.markdown("#### Cloud Remediation Command (AWS CLI)")
        st.code(aws_cmd, language="bash")

```

=====

=====

FILE PATH:

**SentinelAI/dashboard/components/incident_story
line_card.py**

"""

Incident Storyline Component for SentinelAI Dashboard.

Renders an enterprise step-by-step chronological attack timeline using
native

Streamlit UI components to guarantee zero raw HTML tag bleeding or
string overflow.

"""

import streamlit as st

from typing import Dict, Any

def render_incident_storyline_card(storyline_data: Dict[str, Any]):

"""Render step-by-step chronological attack storyline card in Streamlit
using native UI."""

entity_id = storyline_data.get("entity_id", "Unknown")

stages = storyline_data.get("storyline_stages", [])

narrative = storyline_data.get("attack_narrative", "")

highest_sev = storyline_data.get("highest_severity", "LOW")

```
st.markdown("### Chronological Attack Storyline Engine")
```

```
st.caption(f"Entity Sequence Investigation & Evidence Chain Linker • Ta
```

```
# Narrative Summary Box
```

```
if highest_sev == "CRITICAL":
```

```
    st.error(f"**Narrative Summary:** {narrative}")
```

```
elif highest_sev == "HIGH":
```

```
    st.warning(f"**Narrative Summary:** {narrative}")
```

```
else:
```

```
    st.info(f"**Narrative Summary:** {narrative}")
```

```
if not stages:
```

```
    st.caption("No activity stages to display.")
```

```
    return
```

```

st.markdown("##### Step-by-Step Incident Chain:")

# Highlight anomalous / critical stages first, limit main view to top 1
milestones = [s for s in stages if s.get("attack_vector") != "Normal"]
if not milestones:
    milestones = stages[:10] # Take first 10 for normal user baseline

for stage in milestones:
    step_num = stage["step_number"]
    ts = stage["timestamp"]
    title = stage["stage_title"]
    action = stage["action"]
    vector = stage["attack_vector"]
    evidence = stage["evidence_chain"]
    ev_str = " • ".join(evidence) if evidence else "Standard telemetry"

    with st.container():
        st.markdown(f"Step {step_num}: {title} ({ts})")
        st.markdown(f"Action: {action} | Vector: {vector}")
        st.markdown(f"Evidence Chain: {ev_str}")
        st.markdown("----")

# If there are additional baseline events, place them inside a clean ex
if len(stages) > len(milestones):
    with st.expander(f"View All {len(stages)} Telemetry History Milesto
        for stage in stages[len(milestones):]:
            st.markdown(f"Step {stage['step_number']}: {stage['stage_
            st.markdown(f"Action: {stage['action']} | Vector: {stage

```

=====

=====

FILE PATH:

SentinelAI/dashboard/components/behavioral_identity_card.py

"""

Behavioral Identity Card Dashboard Component.

Renders an enterprise visual card displaying employee baseline profiles, deviation percentages, top anomaly drivers, and natural language explanations using native Streamlit components.

```
"""
```

```
import streamlit as st
from typing import Dict, Any
```

```
def render_behavioral_identity_card(evaluation_result: Dict[str, Any]):
    """Render the Behavioral Identity evaluation card in Streamlit using native UI."""
```

```
    entity_id = evaluation_result.get("entity_id", "Unknown")
    dev_pct = evaluation_result.get("deviation_percentage", 0.0)
    summary = evaluation_result.get("behavior_summary", "Baseline Normal")
    explanation = evaluation_result.get("human_explanation", "")
    top_deviations = evaluation_result.get("top_deviations", [])
    baseline = evaluation_result.get("historical_baseline", {})
```

```
    st.markdown("### Behavioral Identity Engine Profile")
```

```
    st.caption(f"Entity Baseline vs Real-Time Access Telemetry Comparison •
```

```
    if dev_pct > 50:
```

```
        st.error(f"**{summary}**\n\n{explanation}")
```

```
    elif dev_pct > 25:
```

```
        st.warning(f"**{summary}**\n\n{explanation}")
```

```
    else:
```

```
        st.info(f"**{summary}**\n\n{explanation}")
```

```
    col1, col2 = st.columns(2)
```

```
    with col1:
```

```
        st.markdown("#### Top Anomaly Deviations")
```

```
        if not top_deviations:
```

```
            st.info("No baseline behavioral deviations detected.")
```

```
        else:
```

```
            for dev in top_deviations:
```

```
                st.markdown(f"- **{dev['factor']}** `[{dev['severity']}]` (d
```

```
                st.markdown(f"  *{dev['detail']}*")
```

```
    with col2:
```

```

st.markdown("#### Entity Baseline Profile Parameters")
st.markdown(f"- **Entity ID:** `{baseline.get('entity_id', entity_i`
st.markdown(f"- **Normal Hours:** `{baseline.get('working_hours', [
st.markdown(f"- **Approved Devices:** `{' '.join(baseline.get('typ
st.markdown(f"- **Approved Countries:** `{' '.join(baseline.get('t
st.markdown(f"- **Frequent Resources:** `{' '.join(baseline.get('n
st.markdown(f"- **Avg Session Duration:** `{baseline.get('avg_sessi

```

```

=====
=====

```

FILE PATH:

SentinelAI/dashboard/components/risk_engine_card.py

```

"""

```

Dynamic Enterprise Risk Card Dashboard Component.

Renders a multi-factor risk scorecard visualization displaying factor weights,

normalized score contributions, velocity multipliers, and recommended actions

using native Streamlit components.

```

"""

```

```

import streamlit as st

```

```

from typing import Dict, Any

```

```

def render_dynamic_risk_card(risk_result: Dict[str, Any]):

```

```

    """Render the Dynamic Enterprise Risk card in Streamlit using native UI."""

```

```

    score = risk_result.get("composite_risk_score", 0.0)

```

```

    tier = risk_result.get("risk_tier", "LOW")

```

```

    multiplier = risk_result.get("velocity_multiplier", 1.0)

```

```

    explanation = risk_result.get("risk_explanation", "")

```

```

    action = risk_result.get("recommended_soc_action", "")

```

```

    breakdown = risk_result.get("factor_breakdown", {})

```

```

st.markdown("#### Dynamic Multi-Factor Enterprise Risk Engine")

```

```

st.caption(f"Normalized 9-Factor Weighted Risk Scoring • Composite Risk

```

```

if tier == "CRITICAL":

```

```

        st.error(f"**Risk Assessment:** {explanation}")
    elif tier == "HIGH":
        st.warning(f"**Risk Assessment:** {explanation}")
    else:
        st.info(f"**Risk Assessment:** {explanation}")

st.markdown("#### 9-Factor Weighted Risk Breakdown")
cols = st.columns(3)

factors_list = list(breakdown.items())
for idx, (fname, fdata) in enumerate(factors_list):
    c = cols[idx % 3]
    pts = fdata["risk_contribution_pts"]
    weight = fdata["weight_pct"]
    score_norm = fdata["score_norm"]
    clean_name = fname.replace("_", " ").title()

    with c:
        st.metric(label=clean_name, value=f"{pts} pts", delta=f"Weight

st.markdown(f"**Recommended Automated Action:** `{action}`")

```

=====

=====

FILE PATH:

SentinelAI/detection/ai_analyst_copilot.py ▶

"""

AI Analyst Copilot Engine for SentinelAI.

Assists SOC analysts by synthesizing raw ML predictions and telemetry into actionable, explainable investigation briefs (Executive Summary, Why Detected, Supporting Evidence, Business Impact, MITRE ATT&CK Mapping, SOC Remediation Commands, and Investigation Savings).

"""

```

from typing import Dict, Any, List, Optional
import pandas as pd

```

```
class AIAlystCopilot:
    """AI Assistant Engine for SOC Incident Investigation and Triage
    Automation."""
```

```
MITRE_MAPPING = {
    "impossible_travel": {
        "tactic": "Initial Access / Defense Evasion",
        "technique_id": "T1078.004",
        "technique_name": "Valid Accounts: Cloud Accounts",
        "url": "https://attack.mitre.org/techniques/T1078/004/",
    },
    "brute_force": {
        "tactic": "Credential Access",
        "technique_id": "T1110.001",
        "technique_name": "Brute Force: Password Guessing",
        "url": "https://attack.mitre.org/techniques/T1110/001/",
    },
    "lateral_movement": {
        "tactic": "Lateral Movement",
        "technique_id": "T1021.002",
        "technique_name": "Remote Services: SMB/Windows Admin Shares",
        "url": "https://attack.mitre.org/techniques/T1021/002/",
    },
    "low_slow_exfiltration": {
        "tactic": "Exfiltration",
        "technique_id": "T1041",
        "technique_name": "Exfiltration Over C2 Channel",
        "url": "https://attack.mitre.org/techniques/T1041/",
    },
    "credential_stuffing": {
        "tactic": "Credential Access",
        "technique_id": "T1110.004",
        "technique_name": "Brute Force: Credential Stuffing",
        "url": "https://attack.mitre.org/techniques/T1110/004/",
    },
}

def __init__(self):
    pass
```

```

def generate_investigation_brief(self, event: Dict[str, Any], risk_data
    """Generate a complete 10-point AI Analyst Investigation Brief for
    entity_id = str(event.get("user_id") or event.get("entity_id") or "
    event_id = str(event.get("event_id", "evt-001"))
    attack_raw = str(event.get("attack_type", "impossible_travel")).low
    dept = str(event.get("department", "Engineering"))
    res = str(event.get("resource_accessed") or event.get("resource_cat
    bytes_tx = float(event.get("bytes_transferred", 0.0))
    geo = str(event.get("geo_location", "Unknown Location"))
    device = str(event.get("device_id", "Unrecognized Device"))

    # Risk score extraction
    composite_risk = risk_data.get("composite_risk_score", 88.5) if ris
    risk_tier = risk_data.get("risk_tier", "CRITICAL") if risk_data els

    # 1. MITRE ATT&CK Mapping
    mitre_info = self.MITRE_MAPPING.get(attack_raw, {
        "tactic": "Persistence / Execution",
        "technique_id": "T1078",
        "technique_name": "Valid Accounts",
        "url": "https://attack.mitre.org/techniques/T1078/",
    })

    # 2. Executive Summary
    exec_summary = (
        f"Anomalous access activity detected for user '{entity_id}' in
        f"Incident flagged as '{attack_raw.replace('_', ' ').title()}'
        f"with a Composite Risk Score of {composite_risk}/100 ({risk_tie
    )

    # 3. Why Detected
    why_detected = (
        f"The anomaly detection ensemble identified significant behavior
        f"Access originated from '{geo}' using device '{device}' outside
        f"Exfiltration volume reached {bytes_tx:,.0f} bytes."
    )

    # 4. Supporting Evidence

```

```

evidence = [
    f"Entity ID: {entity_id} (Department: {dept})",
    f"Source Location: {geo}",
    f"Device Identifier: {device}",
    f"Target Resource: {res}",
    f>Data Transferred: {bytes_tx:,.0f} bytes",
    f"MITRE TTP Alignment: {mitre_info['technique_id']} ({mitre_inf
]

# 5. Risk Score Explanation
risk_explanation = (
    f"Composite Risk Score ({composite_risk}/100) reflects high ass
    f"unrecognized device fingerprinting, and off-hours geographica
)

# 6. Business Impact
if risk_tier == "CRITICAL":
    biz_impact = (
        f"HIGH BUSINESS IMPACT. Potential compromise of confidentialia
        f"unauthorized access to critical infrastructure resource '
    )
else:
    biz_impact = f"MODERATE BUSINESS IMPACT. Requires analyst verif

# 7. Recommended Actions
rec_actions = [
    f"1. Instantly revoke active OAuth/session tokens for user '{en
    f"2. Trigger mandatory out-of-band Multi-Factor Authentication
    f"3. Isolate device '{device}' from corporate internal network
    f"4. Audit recent database queries on resource '{res}' over the
]

# 8. Suggested SOC Response (CLI Script)
powershell_cmd = f"Disable-LocalUser -Name '{entity_id}'; Revoke-Az
aws_cli_cmd = f"aws iam deactivate-mfa-device --user-name {entity_i

# 9. Estimated Investigation Time Savings
est_time_saved = "15-20 minutes saved per analyst investigation"

```

```

# 10. Confidence Level
confidence_pct = 96.0

return {
    "event_id": event_id,
    "entity_id": entity_id,
    "executive_summary": exec_summary,
    "why_detected": why_detected,
    "supporting_evidence": evidence,
    "risk_score_explanation": risk_explanation,
    "business_impact": biz_impact,
    "recommended_actions": rec_actions,
    "mitre_attack_mapping": mitre_info,
    "suggested_soc_response": {
        "powershell": powershell_cmd,
        "aws_cli": aws_cli_cmd,
    },
    "estimated_investigation_time_saved": est_time_saved,
    "copilot_confidence_pct": confidence_pct,
}

```

```

=====
=====

```

FILE PATH: SentinelAI/detection/detector.py

```

"""

```

SentinelAI – Ensemble Anomaly Detector Orchestrator

Blends Isolation Forest (0.4) and PyTorch Autoencoder (0.6)

reconstruction scores

into a unified behavioral anomaly risk score [0, 1]. Converts continuous scores

to risk severity tiers (HIGH, MEDIUM, LOW) based on top alert budget thresholds.

```

"""

```

```

import os

```

```

import logging

```

```

from typing import Dict, Tuple, Optional, List

```

```

import numpy as np
import pandas as pd

from .isolation_forest import IsolationForestModel
from .autoencoder import AutoencoderModel

logger = logging.getLogger(name)

class AnomalyDetector:
    """Ensemble Behavioral Anomaly Detection Engine."""

    def __init__(
        self,
        if_weight: float = 0.4,
        ae_weight: float = 0.6,
        high_percentile: float = 99.0,    # Top 1%
        med_percentile: float = 95.0,    # Top 5%
        random_state: int = 42,
    ):
        self.if_weight = if_weight
        self.ae_weight = ae_weight
        self.high_percentile = high_percentile
        self.med_percentile = med_percentile

        self.if_model = IsolationForestModel(random_state=random_state)
        self.ae_model = AutoencoderModel(random_state=random_state)

        self.high_threshold: Optional[float] = None
        self.med_threshold: Optional[float] = None
        self.is_fitted = False

    def fit(self, X: pd.DataFrame) -> "AnomalyDetector":
        """Fit both Isolation Forest and Autoencoder models."""
        self.if_model.fit(X)
        self.ae_model.fit(X)

        train_scores = self.predict_scores(X)
        self.high_threshold = np.percentile(train_scores, self.high_percentile)
        self.med_threshold = np.percentile(train_scores, self.med_percentile)

        self.is_fitted = True

```

```

        return self

def predict_scores(self, X: pd.DataFrame) -> np.ndarray:
    """Compute weighted continuous anomaly scores in range [0, 1]."""
    if_scores = self.if_model.predict_score(X)
    ae_scores = self.ae_model.predict_score(X)

    return (self.if_weight * if_scores) + (self.ae_weight * ae_scores)

```

```

=====
=====

```

FILE PATH:

SentinelAI/backend/services/incident_persistence.py

```

"""

```

Incident Persistence Service for SentinelAI.

Updates incident status persistently directly in the dataset CSV single source of truth

and logs auditor actions to analyst_actions.csv.

```

"""

```

```

import os
import pandas as pd
from datetime import datetime
from typing import Dict, Any, Optional

```

```

PROJECT_ROOT =
os.path.dirname(os.path.dirname(os.path.dirname(os.path.abspath(file))))
DATASET_PATH = os.path.join(PROJECT_ROOT, "data", "processed",
"engineered_dataset.csv")
AUDIT_LOG_PATH = os.path.join(PROJECT_ROOT, "data", "processed",
"analyst_actions.csv")

```

```

def ensure_persistence_schema(df: pd.DataFrame) -> pd.DataFrame:
    """Ensure dataset has status and analyst triage feedback columns."""
    if "status" not in df.columns:
        df["status"] = df["label"].apply(lambda l: "LIVE_THREAT" if l ==
"anomaly" else "BENIGN")
    if "analyst_feedback" not in df.columns:

```

```

df["analyst_feedback"] = ""
if "last_updated" not in df.columns:
df["last_updated"] = ""
if "updated_by" not in df.columns:
df["updated_by"] = ""
return df

def update_incident_status(event_id: str, new_status: str, analyst_id:
str = "ANALYST-MARK1", remarks: str = "") -> bool:
    """Persistently update incident status in engineered_dataset.csv and
    record audit action."""
    if not os.path.exists(DATASET_PATH):
    return False

```

```

df = pd.read_csv(DATASET_PATH, low_memory=False)
if "event_id" not in df.columns:
    return False

if event_id not in df["event_id"].values:
    return False

# Update status column
df.loc[df["event_id"] == event_id, "status"] = new_status
df.to_csv(DATASET_PATH, index=False)

# Append to analyst audit log
audit_entry = {
    "timestamp": datetime.utcnow().isoformat(),
    "event_id": event_id,
    "analyst_id": analyst_id,
    "action_taken": new_status,
    "remarks": remarks,
}
df_entry = pd.DataFrame([audit_entry])

if os.path.exists(AUDIT_LOG_PATH):
    df_entry.to_csv(AUDIT_LOG_PATH, mode="a", header=False, index=False)
else:
    df_entry.to_csv(AUDIT_LOG_PATH, index=False)

```

```
return True
```

```
def get_audit_log() -> pd.DataFrame:
    """Retrieve analyst actions audit log dataframe."""
    if os.path.exists(AUDIT_LOG_PATH):
        return pd.read_csv(AUDIT_LOG_PATH)
    return pd.DataFrame(columns=["timestamp", "event_id", "analyst_id",
                                "action_taken", "remarks"])
```

```
=====
=====
```

FILE PATH: SentinelAI/backend/app.py

```
"""
SentinelAI FastAPI Production REST API Server

Entry Point for Backend Machine Learning & Incident Intelligence
Services.
"""

from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
import uvicorn

app = FastAPI(
    title="SentinelAI REST API",
    description="Enterprise Behavioral Anomaly Detection & SOC Incident
Intelligence API",
    version="1.0.0",
)

app.add_middleware(
    CORSMiddleware,
    allow_origins=[""],
    allow_credentials=True,
    allow_methods=[""],
    allow_headers=["*"],
)
```

```
@app.get("/")
def root():
    return {"message": "SentinelAI REST API Service Operational", "status":
"active"}

if name == "main":
    uvicorn.run(app, host="0.0.0.0", port=8000)
```